

Q.No:2

Question description

Udaya family is going to order a large pizza and eat it together. A total of n family members will join the order, and there are m possible toppings. The pizza may have any number of toppings.

Each family member gives two wishes concerning the toppings of the pizza. The wishes are of the form "topping x is good/bad". Your task is to choose the toppings so that at least one wish from everybody becomes true (a good topping is included in the pizza or a bad topping is not included).

Input

The first input line has two integers n and m : the number of family members and toppings. The toppings are numbered $1, 2, \dots, m$.

After this, there are n lines describing the wishes. Each line has two wishes of the form "+ x " (topping x is good) or "- x " (topping x is bad).

Output

Print a line with m symbols: for each topping "+" if it is included and "-" if it is not included. You can print any valid solution.

If there are no valid solutions, print "IMPOSSIBLE".

Constraints

- $1 \leq n, m \leq 10^5$
- $1 \leq x \leq m$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
```

EXPECTED OUTPUT

```
- + + + -
```

Test Case 2

INPUT (STDIN)

```
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
```

EXPECTED OUTPUT

```
- + - - +
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void link(int i,int j)
```

Code:

```
#include <stdio.h>

#define MAXN 100005
#define MAXV 200005
#define MAXE 400005

int n, m;

int head[MAXN];
int to[MAXE];
```

```
int next[MAXE];
int edges = 0;

int rhead[MAXV];
int rto[MAXE];
int rnext[MAXE];
int redges = 0;

int visited[MAXV];
int order[MAXV];
int orderSize = 0;

int component[MAXV];
int compCount = 0;
void link(int i, int j)
{
}

void addEdge(int u, int v)
{
    to[edges] = v;
    next[edges] = head[u];
    head[u] = edges++;

    rto[redges] = u;
    rnext[redges] = rhead[v];
    rhead[v] = redges++;
}

void dfs1(int u)
{
    visited[u] = 1;

    for (int e = head[u]; e != -1; e = next[e])
    {
        int v = to[e];

        if (!visited[v])
            dfs1(v);
    }

    order[orderSize++] = u;
}

void dfs2(int u, int c)
{
    component[u] = c;

    for (int e = rhead[u]; e != -1; e = rnext[e])
    {
        int v = rto[e];

        if (component[v] == -1)
```

```
dfs2(v, c);
}
}

int getNode(char sign, int x)
{
    x--;

    if (sign == '+')
        return 2 * x + 1;

    return 2 * x;
}

int opposite(int x)
{
    return x ^ 1;
}

int main()
{
    scanf("%d %d", &n, &m);

    for (int i = 0; i < 2 * m; i++)
    {
        head[i] = -1;
        rhead[i] = -1;

        component[i] = -1;
    }

    for (int i = 0; i < n; i++)
    {
        char s1, s2;
        int x, y;

        scanf(" %c %d %c %d", &s1, &x, &s2, &y);

        int a = getNode(s1, x);
        int b = getNode(s2, y);

        addEdge(opposite(a), b);
        addEdge(opposite(b), a);
    }

    for (int i = 0; i < 2 * m; i++)
    {
        if (!visited[i])
            dfs1(i);
    }

    for (int i = orderSize - 1; i >= 0; i--)
    {
        int v = order[i];
```

```
if (component[v] == -1)
{
    dfs2(v, compCount);
    compCount++;
}
}
for (int i = 0; i < m; i++)
{
    if (component[2 * i] == component[2 * i + 1])
    {
        printf("IMPOSSIBLE\n");
        return 0;
    }
}
printf("Output:\n");
for (int i = 0; i < m; i++)
{
    if (component[2 * i + 1] > component[2 * i])
        printf("+ ");
    else
        printf("- ");
}

printf("\n");

return 0;
}
```

Output:

Test_case:1

```
aadityajaiswal@Aaditya
3 5
+ 1 + 2
- 1 + 3
+ 4 - 2
Output:
- + + + -
aadityajaiswal@Aaditya
```

Test_case:2

```
aadityajaiswal@Aaditya
3 5
+ 2 + 2
- 3 + 3
+ 5 - 2
Output:
- + - - +
aadityajaiswal@Aaditya
```